

Eazy POS

POS BACKEND API REQUIREMENTS

Connector API specification for completing purchases, inventory, reporting, offline synchronization and device-enabled payment workflows.

Version 1.0 | 12 August 2026

Target backend: EazyERP Laravel application

Target clients: Flutter Web, Windows, Android and iOS

Purpose

This document is the implementation handoff for the backend developer. It separates endpoints already available in the OAuth Connector API from functionality that exists only in the session-based Laravel web application, and defines the proposed JSON APIs required by the Flutter POS.

Important

All routes marked PROPOSED are requirements, not existing production endpoints. Existing web routes must not be called directly from Flutter because they depend on Laravel sessions, CSRF, Blade views and DataTables HTML.

1. Executive summary

The EazyERP backend already contains substantial purchase, purchase-order, stock and reporting business logic. However, most of that logic is available only through authenticated Laravel web routes. The Flutter app authenticates with OAuth and consumes routes under /connector/api; therefore it needs dedicated JSON contracts that reuse the existing domain logic without relying on session state or rendered HTML.

Current conclusion

Area	Backend capability	Connector API	Flutter status
Products	Read and write logic exists	Available	Integrated
Customers	Read/create/update exists	Available	Integrated
POS sales	Create/list/update/delete/return exists	Mostly available	Basic sale integrated
Purchase orders	Full web workflow exists	Missing	Cannot implement safely
Purchase bills	Full web workflow exists	Missing	Placeholder
Inventory	Adjustment/transfer/opening stock exists	Read-only report only	Read-only
Reports	Many web reports exist	Only P&L; and stock	Basic
Offline sync	No conflict-safe mobile contract	Missing	Not implemented
Hardware payment	Vendor-specific	Missing	Manual method recording only

Delivery priorities

- P0 - Required for operational purchasing and inventory: purchase orders, purchases, supplier payments, returns, stock adjustments, stock transfers and stock movement history.
- P1 - Required for complete daily operations: cash register correction, advanced sales returns/refunds and detailed reports.
- P2 - Required for resilient multi-device rollout: offline synchronization, idempotency, conflict handling, terminal integration and device auditability.

Recommended approach

Create new API controllers/resources under Modules/Connector. Reuse the existing service/util layers and transaction models, but derive business_id and user_id from Auth::user(), return normalized JSON resources, validate permissions explicitly and wrap every stock/financial mutation in a database transaction.

2. Verified existing Connector API

These routes are already present under the auth:api and timezone middleware group and can be treated as the current baseline.

Method	Endpoint	Purpose	Current Flutter usage
POST	/oauth/token	Password-grant authentication	Login
GET	/connector/api/user/loggedin	Current user and permissions	Shell/settings
GET	/connector/api/business-details	Business configuration	Bootstrap
GET	/connector/api/business-location	Permitted stores	POS/product forms
GET	/connector/api/product	Products, variations, prices, stock	POS/products
POST	/connector/api/products	Create product	Integrated
PATCH	/connector/api/products/{id}	Edit product	Integrated
POST	/connector/api/products/save_quick_product	Quick product and opening stock	Integrated
POST	/connector/api/products/bulk-update	Bulk product update	Integrated
POST	/connector/api/import-products/store	Spreadsheet import	Integrated
GET	/connector/api/taxonomy?type=product	Categories/subcategories	Integrated read-only
GET	/connector/api/unit brand tax	Catalog lookups	Product forms
GET/POST/ PUT	/connector/api/contactapi	Customer/supplier contacts	Customer flow
GET/POST/ PUT/DELETE	/connector/api/sell	Sales CRUD	Create/list currently
POST	/connector/api/sell-return	Sales return	No Flutter UI yet
GET	/connector/api/product-stock-report	Paginated stock report	Inventory read-only
GET	/connector/api/profit-loss-report	P&L; totals	Reports
GET	/connector/api/payment-methods	Configured payment methods	Checkout

Known existing-contract observations

- POST /sell may return HTTP 200 even when an individual item fails; errors are nested in the response body.
- Final sale creation requires purchase/opening-stock allocation records, not only qty_available.
- The cash-register resource declares update, but the verified controller does not implement update().
- Product SKU generation in the web utility reads session state; the Flutter client currently generates a unique SKU for stateless API requests.

3. Required API standards

All proposed endpoint groups should follow one consistent contract. This is essential for clean Flutter repository implementations and future offline synchronization.

Authentication and authorization

- Use auth:api and timezone middleware.
- Obtain user, business_id and permission scope from Auth::user(); never trust business_id supplied by the client.
- Enforce the same permissions used by the Laravel web workflows.
- Filter locations through the authenticated user's permitted_locations().
- Return 401 for invalid/expired tokens and 403 for insufficient permissions.

Contract rules

- Return JSON resources with consistent success, validation and error structures.
- Use ISO 8601 timestamps with timezone.
- Return monetary values consistently as decimal strings or minor-unit integers; do not mix both.
- Paginate all lists and support per_page within a controlled maximum.
- Support updated_since or cursor pagination for sync-capable resources.
- Accept Idempotency-Key on every financial or stock mutation.
- Return resource version or updated_at for optimistic concurrency.
- Never return Blade HTML, DataTables action markup or localized HTML fragments.

Backend ownership

The backend team should define and publish the final field-level payload contracts in OpenAPI or Postman. This handoff focuses on required capabilities, routes, business rules and acceptance criteria.

4. Purchase order API - PROPOSED

The existing PurchaseOrderController already implements listing, creation, viewing, editing, deletion, status updates, shipping status, documents and PDF generation. The Connector implementation should reuse those rules with JSON resources.

GET	/connector/api/purchase-orders Paginated/filterable purchase-order list	P0
POST	/connector/api/purchase-orders Create purchase order	P0
GET	/connector/api/purchase-orders/{id} Detailed order including lines and totals	P0
PATCH	/connector/api/purchase-orders/{id} Edit an order that is not locked/completed	P0
DELETE	/connector/api/purchase-orders/{id} Delete when permitted and safe	P0
PATCH	/connector/api/purchase-orders/{id}/status Update ordered/partial/completed and shipping status	P0
POST	/connector/api/purchase-orders/{id}/convert-to-purchase Create purchase bill from remaining PO quantities	P0
GET	/connector/api/purchase-orders/{id}/pdf Download or retrieve printable PDF	P0

List filters

- location_id, supplier_id, status, shipping_status
- start_date, end_date, search, created_by
- sort, page, per_page
- remaining_only=true for open quantities

Acceptance criteria

- Order totals match EazyERP web calculation rules.
- Supplier, product, variation and location must belong to the authenticated business.
- Remaining quantity is returned per line.
- Completed orders cannot be edited unless the existing permission/business rule allows it.
- Conversion is atomic and cannot exceed remaining quantities.

5. Purchase bill API - PROPOSED

Purchase bills create the purchase allocation records that make stock sellable. This group is therefore a prerequisite for reliable POS inventory.

GET	/connector/api/purchases Paginated/filterable purchase bill list	P0
POST	/connector/api/purchases Create purchase bill and receive stock	P0
GET	/connector/api/purchases/{id} Detailed purchase bill	P0
PATCH	/connector/api/purchases/{id} Edit purchase bill and reconcile stock	P0
DELETE	/connector/api/purchases/{id} Void/delete subject to accounting and stock rules	P0
POST	/connector/api/purchases/{id}/payments Record supplier payment	P0
GET	/connector/api/purchases/{id}/payments List purchase payments	P0
POST	/connector/api/purchases/{id}/attachments Upload invoice/document	P0
GET	/connector/api/purchases/{id}/pdf Printable purchase bill	P0

Critical transaction behavior

Creation/update/deletion must run inside a database transaction and reuse EazyERP stock utilities so variation_location_details, purchase lines, available quantities, ledgers and PO remaining quantities stay consistent. Directly updating qty_available is not acceptable.

6. Purchase returns and suppliers - PROPOSED

GET	/connector/api/purchase-returns List purchase returns	P0
POST	/connector/api/purchase-returns Create return against a purchase	P0
GET	/connector/api/purchase-returns/{id} Return details	P0
PATCH	/connector/api/purchase-returns/{id} Edit return when permitted	P0
DELETE	/connector/api/purchase-returns/{id} Cancel/delete return when safe	P0
POST	/connector/api/purchase-returns/{id}/refunds Record supplier refund	P0

Supplier data

The existing contact API can be reused if it reliably supports type=supplier and returns balances, payment terms and permitted actions. Otherwise extend it with the following read model:

GET	/connector/api/suppliers Searchable supplier lookup with balance and terms	P0
GET	/connector/api/suppliers/{id}/statement Purchases, returns, payments and outstanding balance	P1

Return validation

- Return quantity cannot exceed available purchased quantity after prior returns and sales allocation rules.
- Stock, purchase line returned quantity, supplier balance and refund status update atomically.
- Lot/serial/expiry references must be preserved where enabled.
- Return response must include updated purchase and inventory totals.

7. Inventory management API - PROPOSED

The current Connector API exposes only a stock report. The Laravel web app already includes stock adjustment, stock transfer and opening stock workflows, but they require JSON API wrappers.

Stock adjustments

GET	/connector/api/stock-adjustments List adjustments	P0
POST	/connector/api/stock-adjustments Increase/decrease stock with reason	P0
GET	/connector/api/stock-adjustments/{id} Adjustment details	P0
PATCH	/connector/api/stock-adjustments/{id} Edit when accounting period permits	P0
DELETE	/connector/api/stock-adjustments/{id} Reverse adjustment safely	P0

Stock transfers

GET	/connector/api/stock-transfers List inter-location transfers	P0
POST	/connector/api/stock-transfers Create transfer	P0
GET	/connector/api/stock-transfers/{id} Transfer and line status	P0
PATCH	/connector/api/stock-transfers/{id} Edit status/lines before completion	P0
POST	/connector/api/stock-transfers/{id}/dispatch Dispatch from source	P0
POST	/connector/api/stock-transfers/{id}/receive Receive at destination	P0

Stocktake and movement

POST	/connector/api/opening-stock Set initial stock through EazyERP allocation logic	P0
POST	/connector/api/stocktakes Create a physical stock count	P0
POST	/connector/api/stocktakes/{id}/finalize Create reconciliation adjustment	P0
GET	/connector/api/stock-movements Auditable movement ledger	P0

8. Inventory contracts and controls

Required stock movement information

- Occurrence time, product, variation, SKU and product name
- Location and movement type: opening stock, purchase, sale, return, adjustment, transfer out or transfer in
- Quantity before, quantity change and quantity after
- Cost, source record type, source record ID and reference number
- Creating user and idempotency reference

Inventory acceptance criteria

- No mutation directly edits a summary field without creating auditable transaction lines.
- Negative stock follows business configuration and permission rules.
- Source stock is validated before transfer dispatch.
- Receiving is idempotent; repeated requests cannot double-add stock.
- All queries are business- and location-scoped.
- Concurrent writes use row locking or an equivalent safe strategy.

9. Catalog and advanced sales APIs

Catalog administration - PROPOSED

POST	/connector/api/taxonomy Create category/subcategory	P1
PATCH	/connector/api/taxonomy/{id} Edit category/subcategory	P1
DELETE	/connector/api/taxonomy/{id} Delete only when unused or apply replacement	P1
DELETE	/connector/api/products/{id} Deactivate/delete product safely	P1
PATCH	/connector/api/products/{id}/status Activate/deactivate product	P1
POST	/connector/api/products/check-sku Validate SKU uniqueness	P1
DELETE	/connector/api/products/{id}/image Remove existing image	P1

Optional bilingual catalog fields

To translate all live product/category names, add name_en and name_ar (or a translations JSON object) to products and taxonomies, accept both fields on writes, and return localized_name plus the complete translations object. UI-only translation cannot reliably translate arbitrary backend catalog data.

Advanced sales - existing routes plus proposed helpers

Endpoint	State	Backend work
PATCH /connector/api/sell/{id}	Existing	Verify draft/final update contract
DELETE /connector/api/sell/{id}	Existing	Verify permissions and reversal behavior
POST /connector/api/sell-return	Existing	Normalize errors and document payload
GET /connector/api/list-sell-return	Existing	Confirm pagination/filtering
GET /connector/api/sell/{id}/receipt	Proposed P1	JSON receipt and optional PDF
POST /connector/api/sell/{id}/payments	Proposed P1	Add/settle payment
POST /connector/api/sell/{id}/refund	Proposed P1	Atomic return/refund

10. Cash register and reporting APIs

Cash register - PROPOSED correction

The current Connector resource declares update but the verified CashRegisterController contains index, store and show only. Replace ambiguous resource semantics with explicit operational routes.

POST	/connector/api/cash-register/open Open register with initial cash	P1
GET	/connector/api/cash-register/current Current user's open register	P1
POST	/connector/api/cash-register/{id}/cash-in Record cash added	P1
POST	/connector/api/cash-register/{id}/cash-out Record cash removed	P1
POST	/connector/api/cash-register/{id}/close Close and reconcile	P1
GET	/connector/api/cash-register/{id}/summary Expected/actual totals by payment method	P1

Reports - PROPOSED

GET	/connector/api/reports/sales Sales totals and trends	P1
GET	/connector/api/reports/product-sales Product/category/variation performance	P1
GET	/connector/api/reports/purchases Purchase totals, dues and supplier analysis	P1
GET	/connector/api/reports/taxes Input/output tax summary and details	P1
GET	/connector/api/reports/payments Payment-method and due collections	P1
GET	/connector/api/reports/registers Register session reports	P1
GET	/connector/api/reports/inventory-valuation Cost and selling value	P1
GET	/connector/api/reports/stock-movements Movement and adjustment report	P1
GET	/connector/api/reports/returns Sales and purchase returns	P1

11. Offline synchronization - PROPOSED

Offline support cannot be implemented safely by caching screens alone. The backend must make mutations retry-safe and provide incremental change feeds.

GET	/connector/api/sync/changes?cursor={cursor} Download incremental changes	P2
POST	/connector/api/sync/batch Upload queued mutations	P2
POST	/connector/api/sync/resolve-conflict Explicit conflict resolution	P2

Required synchronization behavior

- Identify the client device and each client operation uniquely.
- Preserve the entity, operation type, occurrence time and base record version.
- Store client operation identifiers with a unique constraint scoped to business/client.
- Return the original success result when an operation is retried.
- Do not create duplicate sales, purchases, payments or stock movements.
- Return a conflict with current server version/state for competing edits.
- Include deletions as tombstones in the change feed.
- Advance the cursor only after a complete, ordered page is consumed.

12. Payment terminals, printing and security

Payment terminals - PROPOSED

POST	/connector/api/terminal/payments Initiate vendor payment	P2
GET	/connector/api/terminal/payments/{id} Poll authoritative status	P2
POST	/connector/api/terminal/payments/{id}/cancel Cancel before capture	P2
POST	/connector/api/terminal/payments/{id}/refund Refund captured payment	P2
POST	/connector/api/terminal/webhook Verified vendor webhook	P2

Terminal rules

- Do not trust a success result sent only by the Flutter device.
- Verify terminal/webhook signatures server-side.
- Store provider transaction ID, masked card details, authorization code and status history.
- Link a captured terminal payment to the sale atomically or through a recoverable pending state.

Printing

Bluetooth, USB, network and A4 printing are primarily Flutter/platform integrations. Backend support is useful for canonical receipt JSON and PDFs, but the backend does not directly control most local printers.

GET	/connector/api/sell/{id}/receipt Canonical receipt data	P1
GET	/connector/api/sell/{id}/receipt.pdf Server-rendered A4/receipt PDF	P2

Authentication security

A client_secret embedded in Flutter Web, Windows, Android or iOS is extractable and must not be treated as confidential. For production, use a backend-for-frontend login proxy or a public-client flow such as Authorization Code with PKCE. Configure explicit CORS origins and never expose broad wildcard credentials policy.

13. Definition of done

A proposed API group is complete only when all applicable items below pass.

Category	Acceptance requirement
Contract	OpenAPI/Postman documentation contains method, URL, auth, fields, filters, examples and errors.
Security	Business isolation, permissions and permitted locations are enforced server-side.
Validation	422 responses use field-addressable errors; foreign IDs are ownership-validated.
Transactions	Financial/stock writes are atomic and roll back fully on error.
Idempotency	Repeated mutation with the same key returns the same result without duplicates.
Concurrency	Stock and version conflicts are deterministic and tested.
Pagination	Lists return stable sorting and pagination metadata.
Audit	created_by, updated_by, timestamps and reference links are retained.
Tests	Feature tests cover success, validation, unauthorized, forbidden, cross-business and rollback cases.
Compatibility	Responses contain JSON only and do not require Laravel session/CSRF/Blade/DataTables.

Required test scenarios

- Create PO, partially convert it to purchase, then complete remaining quantity.
- Receive purchase and verify sellable stock plus purchase allocation.
- Return purchased quantity and verify stock/supplier balance.
- Adjust damaged stock and inspect movement history.
- Transfer stock, retry receive request, and prove no duplicate receipt.
- Run two competing stock mutations and verify safe outcome.
- Retry sale/payment/purchase with identical idempotency key.
- Attempt cross-business and unpermitted-location IDs and receive 403/404.

14. Recommended backend delivery plan

Sprint	Scope	Outcome
A - P0	Purchase orders, purchase bills, supplier payments, purchase returns	Flutter can manage purchasing end to end
B - P0	Adjustments, transfers, opening stock, stocktake, movement history	Inventory becomes operational, not read-only
C - P1	Cash register, sales returns/refunds, receipt contract	Complete daily POS operations
D - P1	Sales, purchases, tax, payment, register and inventory reports	Management reporting
E - P2	Idempotency, change feed, batch sync and conflicts	Safe offline-first operation
F - P2	Terminal gateway/webhooks and printer-ready receipt outputs	Hardware rollout readiness

Backend handoff checklist

- Confirm route names and payload field names before Flutter implementation begins.
- Provide an updated Postman collection or OpenAPI document.
- Provide a dedicated QA database with suppliers, locations, products and opening/purchase stock allocations.
- Provide test OAuth credentials through a secure channel, not source control.
- Document permissions required per endpoint.
- Notify Flutter team of migrations and seed changes.
- Agree on monetary representation and timezone rules.
- Run end-to-end QA using the Connector API, not the Laravel web UI.

Source verification references

- Connector API routes: Eazyerp/Modules/Connector/Routes/api.php
- Laravel web routes: Eazyerp/routes/web.php
- Purchase-order implementation: Eazyerp/app/Http/Controllers/PurchaseOrderController.php
- Purchase implementation: Eazyerp/app/Http/Controllers/PurchaseController.php
- Stock workflows: StockAdjustmentController, StockTransferController and OpeningStockController
- Flutter integration checklist: docs/api_integration_checklist.md

Final clarification

Purchase orders, purchase bills and inventory management are present in EazyERP. The missing layer is a stable OAuth-authenticated Connector JSON API for Flutter. The proposed work should wrap and reuse existing domain logic rather than duplicate it.